



**FACULTAD
DE INGENIERIA**

Universidad de Buenos Aires

TRABAJO FINAL INTEGRADOR

Taller de Comunicaciones Digitales (TA137)

1°C 2026

Grupo 2		
Barrionuevo, Juan Bautista	109086	jbbarrionuevo@fi.uba.ar
Gamberale, Mercedes	108834	mgamberale@fi.uba.ar
Leon, Martín	109138	mfleon@fi.uba.ar
Ochoa, Amalia	107129	amochoa@fi.uba.ar

1. Introducción

El presente trabajo práctico tiene como objetivo simular y analizar de manera integral un sistema de comunicaciones digitales capaz de transmitir un archivo de texto desde un extremo transmisor hasta un extremo receptor. Para ello, se implementa una cadena completa de procesamiento que permite representar la información original, adaptarla para su transmisión, modelar los efectos del canal y reconstruir el mensaje recibido para compararlo con el texto de entrada.

El desarrollo se organiza de forma modular, siguiendo la estructura conceptual de un sistema de comunicaciones compuesto por transmisor, canal y receptor. En el transmisor se incluyen las etapas de codificación de fuente, codificación de canal y modulación. En el canal se consideran efectos tales como ruido aditivo blanco gaussiano (AWGN) e interferencia intersimbólica (ISI), mientras que en el receptor se implementan los procesos inversos necesarios para recuperar la información transmitida: demodulación, decodificación de canal y decodificación de fuente.

La implementación se realiza en **MATLAB**, utilizando funciones separadas para cada bloque del sistema. Esta organización permite verificar progresivamente el funcionamiento de cada etapa y facilita el análisis posterior del desempeño ante distintas condiciones de transmisión. En particular, se estudian técnicas de codificación de fuente, modulaciones en banda base y banda pasante, y códigos lineales de bloques para detección y corrección de errores.

El informe presenta la metodología empleada, las funciones implementadas, los parámetros utilizados y los resultados obtenidos en cada módulo. A partir de las simulaciones realizadas, se busca evaluar la calidad de recuperación del texto transmitido y analizar cómo influyen la modulación, el ruido, el canal y la codificación en el comportamiento global del sistema.

2. Programa principal: datos, control y estructura general del sistema

En esta sección se va a presentar la estructura general del simulador desarrollado para transmitir un archivo de texto a través de un sistema de comunicaciones digital compuesto por transmisor, canal y receptor. Se simulan la codificación y decodificación de las fuentes, por medio de la creación e implementación de un diccionario de Huffman en función de la probabilidad de aparición de los caracteres. También se emulan la codificación y decodificación del canal mediante un código de Hamming, que permite detectar y corregir errores en los datos transmitidos. Por otro lado, se simulan los efectos de la modulación y demodulación para enviarlos por el canal. Finalmente, los efectos del canal se podrán analizar por medio de la simulación, tanto el ruido térmico como la atenuación de señal al propagar la misma; a pesar de que ésta última no se aplicará a la señal.

2.1. Diagrama del sistema Implementado

El diagrama del sistema implementado se presenta en la Figura 1, donde en el transmisor los datos de entrada ingresan a la codificación de fuente, luego pasan por la codificación de canal y finalmente por la modulación y luego ingresan al canal de transmisión. Los datos ingresan al receptor, donde primero se demodula, luego se decodifica el canal y se concluye por decodificar la fuente para recuperar así el mensaje original.

Los bloques funcionales que se implementarán a lo largo de este proyecto se dividen entonces de la siguiente manera:

1. Datos y control;
2. Transmisor;
3. Canal;

4. Receptor.

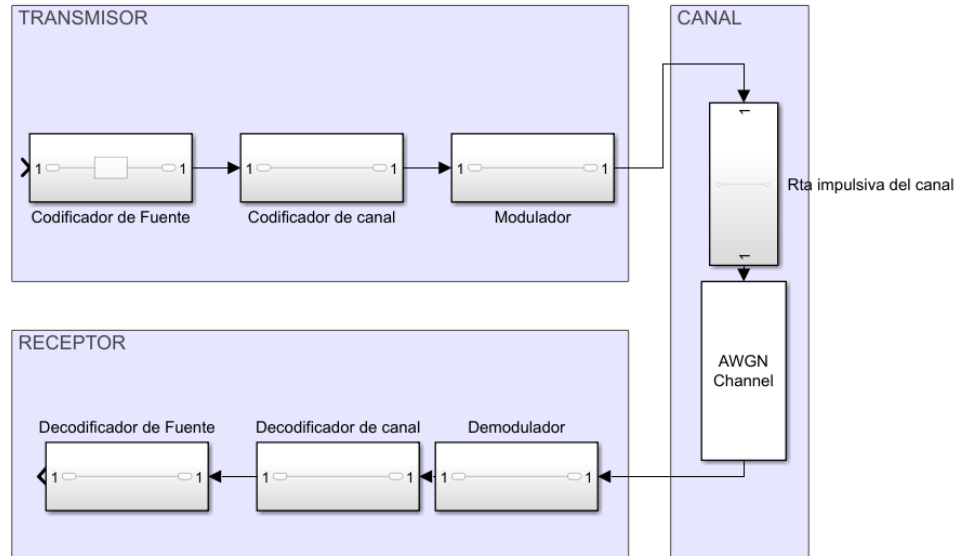


Figura 1: Diagrama en bloques realizado en Simulink (MATLAB)

2.2. Transmisor

El programa principal llama, por medio del bloque transmisor, a los módulos codificador de fuente, codificador de canal y modulador; en ese orden.

El transmisor recibe la información a enviar por el canal y, dentro del bloque de codificación de fuente, crea un diccionario de Huffman asignando distintos símbolos a cada caracter en función de su probabilidad de ocurrencia en los datos a transmitir. Luego, por medio del bloque de codificación del canal, se añade información a modo de poder detectar y corregir errores de bits luego en la recepción de los datos. Finalmente, en la modulación se modularán los datos codificados por fuente y por canal en banda base o banda pasante, por medio de los distintos esquemas de modulación asignados. Los distintos módulos o bloques a realizar en la etapa transmisora se pueden visualizar en la Figura 2.

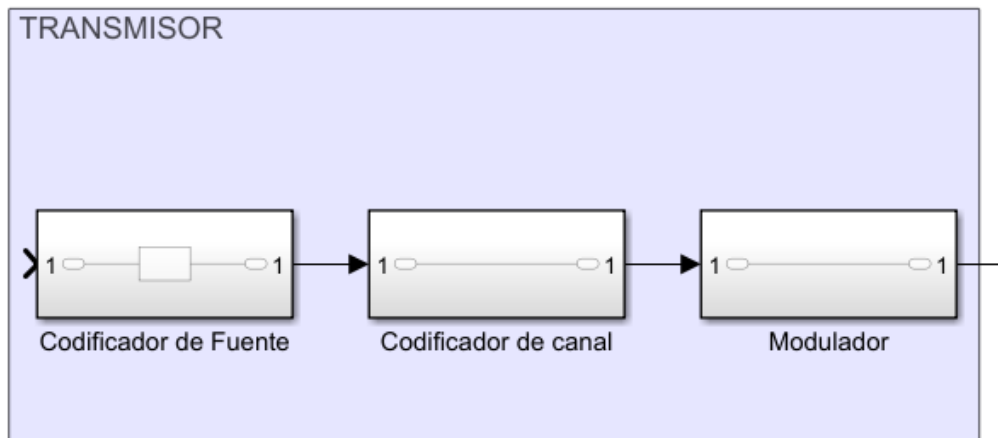


Figura 2: Diagrama en bloques de la etapa transmisora realizado en Simulink (MATLAB).

2.3. Canal

En el canal se encapsulan el modelo de canal utilizado, la simulación de ruido AWGN y la respuesta impulsiva. Aquí es donde entrarán en juego efectos propios del canal como lo son el ruido térmico, la atenuación de la señal o el desvanecimiento selectivo de frecuencias, entre otros. Se generará una muestra aleatoria de ruido térmico en función a una distribución gaussiana. También se generará una atenuación aleatoria del canal. Estos efectos se aplicarán a la señal enviada por el transmisor y finalmente se enviarán al receptor. Los bloques de la etapa de efectos del canal se pueden vislumbrar en la Figura 3.

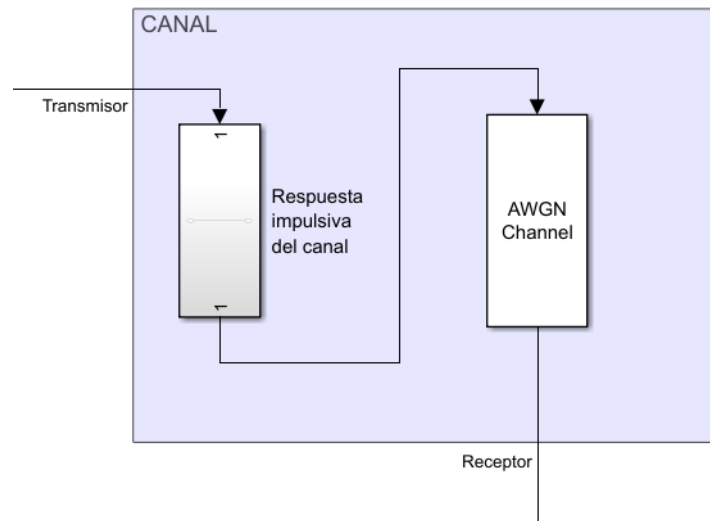


Figura 3: Diagrama en bloques de la etapa de efectos del canal realizado en Simulink (MATLAB).

2.4. Receptor

Dentro del receptor se implementan los módulos de demodulación, decodificación de canal y decodificación de fuente, hasta obtener la entrada reconstruida.

El receptor recibe la información enviada por parte del transmisor por medio del canal, e inicia por demodularla usando el esquema de banda base o banda pasante utilizado en la modulación, de modo de obtener la información enviada en términos de ceros y unos. Luego la información ingresa a la decodificación del canal, donde se verifica que no haya habido errores generados en la transmisión de la información y, de hallarlos, los corrige para enviar información lo más fidedigna a la transmitida por la fuente al decodificador de fuente. Éste se encargará de, utilizando el diccionario de Huffman previamente creado, convertir esa serie de símbolos en los caracteres del mensaje originalmente transmitido.

Los distintos módulos o bloques a realizar en la etapa transmisora se pueden visualizar en la Figura 4.

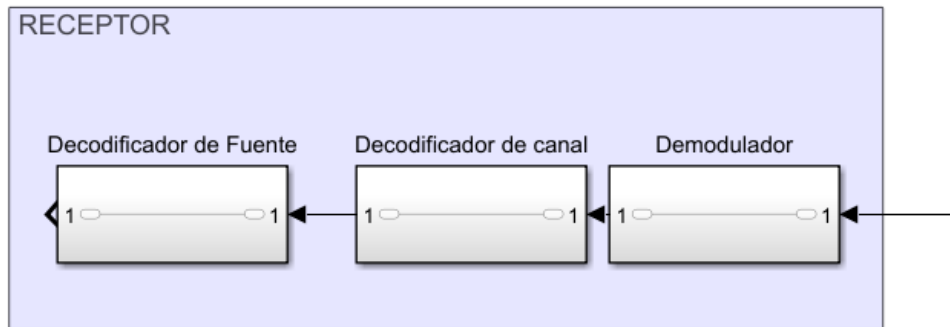


Figura 4: Diagrama en bloques de la etapa receptora realizado en Simulink (MATLAB).

2.5. Verificación inicial del sistema

Una vez realizados todos los bloques del proyecto, se verificará que el comportamiento del sistema sea el deseado por medio de comparaciones entre los mensajes originalmente enviados y finalmente recibidos; y por medio de cálculos de probabilidad de error de símbolo y de bit utilizando pruebas de volumen y reiteradas iteraciones del proyecto completo y modularmente.

3. Codificación y decodificación de fuente

En esta sección se implementa la etapa de codificación y decodificación de fuente del sistema de comunicaciones. El objetivo es comprimir la información contenida en un archivo de texto mediante un código de Huffman, asignando palabras binarias más cortas a los caracteres más probables y palabras más largas a los caracteres menos probables.

La implementación desarrollada toma como entrada el archivo `entrada.txt`, analiza estadísticamente sus caracteres, construye el diccionario de Huffman correspondiente, genera un archivo codificado llamado `salida.txt` y luego reconstruye el texto mediante una etapa de decodificación, guardando el resultado en `salida.decodificacion.txt`.

3.1. Lectura del archivo y obtención de probabilidades

Para la lectura del archivo, se llama a la función `abrir_archivo_lectura()` que toma un nombre conocido como parámetro y realiza la lectura. Una vez abierto el archivo, se invoca la función `leer_archivo()`. Esta función recorre el archivo carácter por carácter y como salida se obtienen el vector `caracteres[]`, donde se registran todos los caracteres distintos, el vector `probabilidades[]`, con las probabilidades asociadas, y `cantidad_caracteres_distintos`, la cantidad de caracteres distintos de la fuente.

Al finalizar la lectura, la función calcula el vector de probabilidades dividiendo la cantidad de apariciones de cada carácter por la cantidad total de caracteres del archivo:

$$p_i = \frac{N_i}{N} \quad (1)$$

donde N_i es la cantidad de apariciones del carácter i y N es la cantidad total de caracteres leídos.

Luego se utiliza la función `verificacion_suma`, que suma todas las probabilidades obtenidas. Esta verificación permite comprobar que la distribución estadística fue construida correctamente, ya que la suma de probabilidades debe ser igual a uno, salvo posibles diferencias numéricas por redondeo.

3.2. Cálculo de la entropía de la fuente

Con el vector de probabilidades obtenido, el programa calcula la entropía de la fuente llamando a la función `calcular_entropia_fuente()`. La entropía se calcula según:

$$H = \sum_i p_i \log_2 \left(\frac{1}{p_i} \right) \quad (2)$$

o, de forma equivalente,

$$H = - \sum_i p_i \log_2(p_i) \quad (3)$$

La entropía representa la cantidad media de información por símbolo de la fuente. En el contexto de la codificación de Huffman, constituye una referencia teórica para evaluar qué tan eficiente resulta la longitud promedio del código obtenido. Cuanto más cercana sea la longitud promedio del código a la entropía, mayor será la eficiencia de la codificación.

3.3. Construcción del diccionario de Huffman

A partir de los caracteres detectados y sus probabilidades, se construye el diccionario de Huffman mediante la función `huffmandict()`. Esta función recibe como entrada el vector `caracteres[]` y el vector `probabilidades[]`, y devuelve dos resultados principales:

- `dict`: diccionario que asocia cada carácter de la fuente con su palabra binaria de Huffman;

- `avglen`: longitud promedio del código generado.

El diccionario tiene dos columnas: en la primera se almacena el símbolo original y en la segunda la secuencia binaria asignada por el algoritmo de Huffman. La asignación resultante depende de la probabilidad de cada carácter: los símbolos más frecuentes reciben palabras de código más cortas, mientras que los menos frecuentes reciben palabras más largas.

3.4. Longitud mínima, longitud promedio y eficiencia

Luego de construir el diccionario, el código calcula dos magnitudes para analizar el desempeño de la codificación. La primera es la longitud mínima teórica, que en la implementación se toma igual a la entropía de la fuente:

$$L_{\text{mín}} = \frac{H}{\log_2(2)} \quad (4)$$

La segunda es la longitud promedio del código de Huffman, entregada por `huffmandict` como `avglen`. Conceptualmente, esta longitud puede escribirse como:

$$\bar{L} = \sum_i p_i l_i \quad (5)$$

donde l_i es la longitud de la palabra de código asignada al carácter i .

Finalmente, se calcula la eficiencia de la codificación como:

$$\eta = \frac{H}{\bar{L}} \quad (6)$$

Este cociente permite comparar la codificación obtenida con el límite teórico dado por la entropía. Un valor de eficiencia cercano a uno indica que el código generado aprovecha adecuadamente la distribución estadística de la fuente.

3.5. Codificación del archivo de texto

La codificación del texto se realiza mediante la función `codificar_archivo`. Esta función abre nuevamente el archivo original y crea un archivo de salida en modo escritura. Luego recorre el texto carácter por carácter y, para cada símbolo leído, busca su posición dentro del diccionario de Huffman.

Una vez encontrada la coincidencia, la palabra binaria asociada se convierte a una cadena de caracteres mediante `sprintf`. Esa cadena, compuesta por los caracteres `0` y `1`, se escribe en el archivo codificado usando `fwrite`. De esta manera, el archivo `salida.txt` contiene la secuencia binaria resultante de reemplazar cada carácter del texto original por su palabra de código correspondiente.

El programa detecta la finalización del archivo de texto original ya que, al inicio de la codificación de Huffman, en el arreglo se incluye un *header* de la cantidad de caracteres totales del archivo original, codificadas en `int64`, 3 veces seguidas. Se repite tres veces para recuperar efectivamente la cantidad de caracteres en caso de ruido excesivo en el canal.

3.6. Decodificación del archivo

La etapa de decodificación se implementa mediante la función `decodificar_archivo`. En esta etapa, el archivo codificado `chau.txt` se toma como entrada y se genera como salida el archivo `salida_decodificacion.txt`.

La función primero verifica la cantidad de caracteres totales a leer con el *header* del arreglo a decodificar, verificando por mejor de 3 bit a bit para corregir errores posibles. Luego lee el archivo codificado carácter por carácter y va acumulando los bits leídos en la variable `simbolo_codificado`.

En cada iteración, esta cadena parcial se compara con las palabras binarias almacenadas en la segunda columna del diccionario de Huffman. Cuando se encuentra una coincidencia, se identifica el carácter original correspondiente, almacenado en la primera columna del diccionario, y se escribe en el archivo de salida.

Luego de escribir el carácter decodificado, la variable `simbolo_codificado` se reinicia para comenzar la búsqueda de la siguiente palabra de código. Este procedimiento funciona porque el código de Huffman es instantáneo o libre de prefijos: ninguna palabra válida es prefijo de otra, por lo que cada secuencia puede decodificarse de manera no ambigua al encontrar una coincidencia en el diccionario.

Además, se implementa un chequeo final `verificar_archivos()` donde se lee `entrada.txt` y `salida_decodificacion.txt` y se verifica que sean iguales.

3.7. Verificación de la implementación

Para verificar el correcto funcionamiento del bloque, aunque la función `verificar_archivos()` confirma automáticamente que ambos archivos son iguales, también se comparó manualmente el contenido de un archivo original de prueba con el contenido del archivo decodificado.

Texto de entrada usado en la prueba

aus Esta es una prueba

La idea de usar esta frase es que las letras `a`, `u` y `s` aparecen varias veces, lo que permite identificar visualmente sus palabras binarias dentro del archivo codificado. Al segmentar la secuencia obtenida con el diccionario de Huffman, se identificaron las siguientes asignaciones:

Códigos identificados en el diccionario de Huffman

`a` → 001 `u` → 011 `s` → 010

Para esta prueba, la secuencia codificada fue:

Secuencia codificada con colores

00101101000011110101110001000110010000111001001000100010110111101010001

En esta visualización se usa `naranja fuerte` para la `a`, `magenta` para la `u` y `naranja suave` para la `s`. De esta manera, podemos identificar que esas 3 letras fueron codificadas correctamente, y sirve para distinguir que la `E` y la `e` se corresponden a distintas codificaciones. Luego de aplicar la función de decodificación, el archivo `salida_decodificacion.txt` recuperó el mismo mensaje textual que el archivo original. Se observa que el contenido textual recuperado coincide con el mensaje original por lo que se puede decir que el sistema implementado funciona correctamente.

Texto de salida usado en la prueba

aus Esta es una prueba

3.8. Validación y Resultados

Para presentar resultados sobre un texto más extenso, se utilizó el mensaje de bienvenida del aula virtual de la materia. Se utiliza este mensaje por tener caracteres especiales dentro de ASCII, números, mayúsculas y minúsculas. De esta manera, se presentan diversos símbolos para validar el sistema.

Texto de entrada usado en la validación

Aula Virtual de la asignatura "TA137 - Taller de Comunicaciones Digitales" ("86.25 - Comunicaciones Digitales I", para aquellos alumnos del Plan 2009).

Docentes: Lunes: Profesor Dr. Ing. Gustavo Abraham Hirchoren Miércoles: JTP Mg. Ing. Leonel Enrique Hochman

Bibliografía sugerida:

Bernard Sklar. Digital Communications: Fundamentals and Applications. Prentice Hall, 2001. John

G. Proakis. Digital Communications. McGraw Hill, 5ta edición, 2008. Notas de apoyo: John Cioffi.

<https://cioffi-group.stanford.edu/> Comunicaciones analógicas: B. P. Lathi. Introducción a la Teoría y

Sistemas de Comunicación. Ed. Limusa, 2005. F. G. Stremler, Introducción a los sistemas de comunicación.

Sincronización: Ph.D Floyd M. Gardner. Phase-lock Techniques. John Wiley and Sons, third edition, 2005.

- Para la aprobación de la cursada se debe aprobar un Proyecto integrador de simulación con software System View, Matlab, Octave, Python u otros, o bien componentes electrónicos modulares. Al finalizar el Proyecto, los alumnos deberán realizar una defensa individual del mismo. La nota de aprobación es de 4 (cuatro) / 10 (diez).

- La materia se aprobará mediante un examen Final integrador y escrito, el cual abarcará todo el programa de contenidos y consistirá en la resolución de ejercicios prácticos y desarrollo de conceptos teóricos, demostraciones y justificaciones. Para aprobar será necesario alcanzar, como mínimo, el 40% del puntaje de cada ejercicio.

A partir de este archivo de validación se obtuvieron los siguientes valores estadísticos y de desempeño para la codificación de Huffman:

Resultados de la validación

- Entropía de la fuente: $H = 4,78497$.
- Longitud mínima: $L_{\min} = 4,78497$.
- Cantidad de caracteres distintos: 70.
- Longitud promedio: $\bar{L} = 4,81406$.
- Eficiencia: $\eta = 0,99396$.

Se observa que la longitud promedio del código de Huffman no es exactamente igual a la entropía de la fuente, aunque ambas magnitudes resultan muy cercanas: $H = 4,784968$ y $\bar{L} = 4,814064$. Por ese motivo, la eficiencia no es exactamente uno, sino $\eta = H/\bar{L} = 0,99396$. Esto indica que el código construido aprovecha muy bien la distribución estadística de los caracteres del texto de validación, aunque todavía presenta una pequeña redundancia respecto del límite teórico dado por la entropía. A continuación se presenta la lista completa de símbolos detectados, las palabras de código generadas por Huffman y sus probabilidades, cuya sumatoria da 1. Se observa también que el código obtenido es prefijo, es decir, que ninguna de sus palabras es prefijo de otra. Viendo en detalle la tabla a continuación, podemos ver que las palabras con más probabilidad de ocurrencia tienen menor longitud de código de Huffman. Por ejemplo, el caracter **Espacio** tiene una probabilidad asociada de 0,1467 y una longitud de código de 3 dígitos. En contraste, el caracter **%**, que apareció una sola vez, tiene una probabilidad de ocurrencia de 0,0007 y una longitud de código de Huffman de 11 dígitos. Esto se condice con la lógica de codificación de Huffman.

Caracteres, probabilidades y códigos de Huffman de la Validación

Caracter	ASCII	Prob.	Código Huffman	Caracter	ASCII	Prob.	Código Huffman
A	65	0.0034	10011110	P	80	0.0088	0110001
u	117	0.0250	10111	O	48	0.0081	0110111
l	108	0.0379	01000	9	57	0.0007	00000110100
a	97	0.0798	0001	)	41	0.0020	011010001
espacio	32	0.1467	001	salto línea	10	0.0135	101100
V	86	0.0014	110111100	:	58	0.0061	1101110
i	105	0.0629	1000	L	76	0.0041	10011001
r	114	0.0507	1100	f	102	0.0074	1001101
t	116	0.0352	01001	G	71	0.0027	000001001
d	100	0.0318	10010	v	118	0.0020	011010000
e	101	0.0690	0101	b	98	0.0081	0110110
s	115	0.0433	1111	h	104	0.0088	0110000
g	103	0.0108	0000101	H	72	0.0027	000001000
n	110	0.0568	1010	M	77	0.0034	10110110
"	34	0.0027	000001111	é	233	0.0007	00000110111
T	84	0.0034	10110101	J	74	0.0027	000001011
1	49	0.0020	100111110	E	69	0.0014	110111110
3	51	0.0007	1101111011	B	66	0.0020	011010011
7	55	0.0007	1101111010	í	237	0.0020	011010010
-	45	0.0034	10110100	S	83	0.0041	10011000
C	67	0.0047	01100100	k	107	0.0020	100111001
o	111	0.0629	0111	F	70	0.0027	000001010
m	109	0.0216	000000	w	119	0.0020	100111000
c	99	0.0439	1110	ó	243	0.0088	0000111
D	68	0.0047	00001101	N	78	0.0007	00000110110
(	40	0.0020	011010111	y	121	0.0081	0110011
8	56	0.0014	110111111	/	47	0.0027	000011001
6	54	0.0007	00000110101	z	122	0.0034	10011101
.	46	0.0237	11010	W	87	0.0007	00000110001
2	50	0.0041	01101010	0	79	0.0007	00000110000
5	53	0.0027	000001110	á	225	0.0041	01100101
I	73	0.0034	10110111	4	52	0.0014	100111111
,	44	0.0115	110110	x	120	0.0007	00000110011
p	112	0.0108	0000100	j	106	0.0027	000011000
q	113	0.0020	011010110	%	37	0.0007	00000110010

4. Modulación y demodulación en banda pasante

La modulación y demodulación en banda pasante constituyen el último bloque del transmisor y el primer bloque del receptor. Su función es adaptar la información digital al canal físico mediante señales portadoras. Para modular correctamente, el codificador de canal debe agrupar la secuencia binaria en palabras de $k = \log_2(M)$ bits, donde M es la cantidad de símbolos de la constelación, de forma tal que a la entrada del modulador haya una cantidad de bits múltiplo de k . En este trabajo se estudiarán las modulaciones M -PSK (*Phase Shift Keying*) y M -FSK (*Frequency Shift Keying*).

4.1. Caracterización de las modulaciones M -PSK y M -FSK

Antes de avanzar con la implementación conviene comparar las expresiones teóricas de energía y probabilidad de error para las dos modulaciones asignadas. En la tabla 4.1 se resumen las magnitudes principales en función del orden M , de la distancia mínima entre símbolos d y de la densidad espectral de ruido N_0 . En esta primera comparación se escriben las expresiones generales sin asumir codificación de Gray para PSK, luego se analiza cómo cambia la interpretación de P_b al usar Gray y cómo se compara con el caso de FSK ortogonal.

Comparación de las modulaciones asignadas por energía media por símbolo y bit, y probabilidad de error por símbolo y por bit

Modulación	M -PSK	M -FSK ortogonal
Energía media por símbolo $\bar{E}_s$	$\frac{d^2}{4 \sin^2(\pi/M)}$	$\frac{d^2}{2}$
Energía media por bit $\bar{E}_b$	$\frac{\bar{E}_s}{\log_2(M)}$	$\frac{\bar{E}_s}{\log_2(M)}$
Probabilidad de error de símbolo P_e	$\approx 2Q\left(\sqrt{\frac{2\bar{E}_s}{N_0}} \sin\left(\frac{\pi}{M}\right)\right)$	$\lesssim (M-1)Q\left(\sqrt{\frac{\bar{E}_s}{N_0}}\right)$
Probabilidad de error de bit P_b	$\approx \frac{\bar{n}_b}{\log_2(M)} P_e$	$\approx \frac{M}{2(M-1)} P_e$

La expresión de P_e para M -PSK incluida en la tabla corresponde al caso $M > 2$, donde se consideran dos vecinos más cercanos para cada símbolo. Para $M = 2$ hay un único vecino posible, por lo que el factor 2 se reemplaza por 1:

$$P_e \approx Q\left(\sqrt{\frac{2\bar{E}_s}{N_0}}\right). \quad (7)$$

En la tabla, $\bar{n}_b$ representa la cantidad media de bits erróneos producidos cuando se decide un símbolo incorrecto. Por eso, para M -PSK sin imponer Gray, no puede afirmarse directamente que $P_b = P_e / \log_2(M)$: esa relación depende del etiquetamiento de los símbolos. Si se usa un mapeo binario natural, un error hacia un símbolo vecino puede cambiar más de un bit, por lo que $\bar{n}_b$ puede ser mayor que uno y la probabilidad de error de bit aumenta.

Al aplicar codificación de Gray en M -PSK, los símbolos vecinos difieren en un único bit. Como a relaciones señal a ruido moderadas o altas los errores más probables son hacia vecinos inmediatos, se toma $\bar{n}_b \approx 1$ y entonces

$$P_b \approx \frac{P_e}{\log_2(M)}. \quad (8)$$

Esta es la principal ventaja de Gray: no modifica la energía de la constelación ni la probabilidad de error de símbolo, pero reduce la cantidad de bits afectados por cada error de símbolo.

Para M -FSK ortogonal, en cambio, los símbolos se representan mediante señales ortogonales en frecuencia. Esto permite separar mejor los símbolos en el espacio de señales, aunque requiere mayor ancho de banda al aumentar M . La expresión de P_b de la tabla se obtiene considerando que, cuando se detecta un símbolo equivocado, el símbolo decidido puede ser cualquiera de los otros $M - 1$ símbolos y el número medio de bits distintos entre dos palabras binarias es aproximadamente $M \log_2(M)/(2(M - 1))$. Por eso resulta

$$P_b \approx \frac{M}{2(M - 1)} P_e. \quad (9)$$

Así, la comparación muestra que Gray es especialmente útil en PSK porque adapta el etiquetamiento a los errores hacia vecinos cercanos, mientras que en FSK ortogonal la comparación se basa principalmente en la separación ortogonal entre señales y en el costo de ancho de banda.

Desde el punto de vista energético, M -FSK ortogonal tiene una ventaja importante: para una distancia mínima d fijada, la energía media por símbolo es $\bar{E}_s = d^2/2$, independiente de M . Como cada símbolo transporta $\log_2(M)$ bits, al aumentar M la energía media por bit disminuye según $\bar{E}_b = \bar{E}_s / \log_2(M)$. Es decir, en FSK ortogonal no aumenta la energía al aumentar el número de símbolos; lo que aumenta es la cantidad de señales ortogonales necesarias. El costo de esta mejora energética es el ancho de banda, ya que se requieren más frecuencias separadas para mantener la ortogonalidad entre símbolos.

En M -PSK, en cambio, todos los símbolos comparten la misma frecuencia y se diferencian por fase, por lo que la modulación es más eficiente espectralmente. La desventaja es que, al aumentar M , los puntos de la constelación quedan más cercanos angularmente y aumenta la probabilidad de confundir símbolos vecinos. De manera equivalente, para un M fijo, aumentar la energía de símbolo agranda la constelación y mejora la distancia mínima entre puntos, disminuyendo tanto P_e como P_b .

En resumen, M -FSK ortogonal ofrece mejor desempeño energético a costa de mayor ancho de banda, mientras que M -PSK aprovecha mejor el espectro pero necesita suficiente relación señal a ruido para mantener separados los símbolos. La elección entre ambas modulaciones depende entonces del compromiso entre potencia disponible, ancho de banda permitido y probabilidad de error objetivo.

4.2. Modulador y demodulador

Para implementar las modulaciones asignadas se programó la función `[simbolosModulados,simbolosOriginales,constelacion]=modularSimbolos(entrada,entradaM,mapeoTipo,modulacionTipo)`. Esta función recibe la secuencia binaria de entrada, el orden de modulación M indicado por `entradaM`, el tipo de mapeo seleccionado (Gray o binario) y la modulación a utilizar (M -PSK o M -FSK). A partir de estos parámetros, agrupa los bits en palabras de $k = \log_2(M)$ bits y devuelve los símbolos modulados, los símbolos originales asociados a cada palabra binaria y la constelación utilizada.

La función también verifica que la cantidad de bits de `entrada` sea múltiplo de k y que M sea una potencia de dos dentro del rango permitido, entre 2 y 16. De esta manera se evita modular secuencias incompletas o utilizar constelaciones no contempladas en el trabajo.

Para la recepción se programó la función `[bitsDetectados,simbolosDetectados]=demodularSimbolos(transmisionRuidosa,entradaM,mapeoTipo,modulacionTipo)`. Esta realiza el proceso inverso: compara cada muestra recibida con los símbolos posibles de la constelación, decide cuál fue el símbolo transmitido más probable y reconstruye la secuencia binaria correspondiente según el mapeo elegido.

4.3. Estimación de energías medias de símbolo y de bit

Para estimar las energías medias a partir de los símbolos obtenidos en la simulación se utilizó la función `[eSimbolo,eBit]=calcularEnergias(simbolosModulados,M)`. Esta función calcula primero la cantidad de bits por símbolo,

$$k = \log_2(M), \quad (10)$$

y luego obtiene la energía de cada símbolo como la norma al cuadrado de sus componentes. Si `simbolosModulados` tiene un símbolo por fila, la energía del símbolo i se calcula como

$$E_{s,i} = \sum_j s_{i,j}^2. \quad (11)$$

En MATLAB esto se implementó mediante `sum(simbolosModulados.^2, 2)`, de modo que la suma se realiza por filas. Luego, la energía media por símbolo se obtiene promediando todos esos valores:

$$\bar{E}_s = \frac{1}{N_s} \sum_{i=1}^{N_s} E_{s,i}. \quad (12)$$

Finalmente, como cada símbolo representa k bits, la energía media por bit se estima como

$$\bar{E}_b = \frac{\bar{E}_s}{k}. \quad (13)$$

De esta manera, las energías estimadas dependen directamente de los símbolos realmente generados por el modulador y pueden compararse con los valores teóricos de cada constelación.

4.4. Cálculo de probabilidad de error de símbolo del sistema

Para estimar la tasa de error de símbolo se utilizó la función `SER=errorSimbolo(simbolosTransmitidos,simbolosDemodulados)`. La función compara los símbolos transmitidos y demodulados fila por fila. Para cada fila calcula la suma de las diferencias absolutas entre componentes:

$$d_i = \sum_j |s_{i,j} - \hat{s}_{i,j}|. \quad (14)$$

Se considera que el símbolo i fue detectado incorrectamente cuando $d_i > 0,001$. Esta tolerancia evita contar como errores pequeñas diferencias numéricas propias del cálculo computacional. Luego se cuenta la cantidad total de símbolos erróneos y se estima

$$SER = \frac{N_{s,err}}{N_s}, \quad (15)$$

donde $N_{s,err}$ es el número de símbolos con error y N_s es la cantidad total de símbolos modulados transmitidos.

4.5. Cálculo de probabilidad de error de bit del sistema

Para estimar la tasa de error de bit se utilizó la función `BER=errorBit(bitsTransmitidos,bitsRecibidos)`. Esta función compara directamente los vectores binarios transmitido y recibido, posición por posición, y cuenta los bits distintos:

$$N_{b,err} = \sum_i \mathbf{1}\{b_i \neq \hat{b}_i\}. \quad (16)$$

Finalmente, la tasa de error de bit se calcula como

$$BER = \frac{N_{b,err}}{N_b}, \quad (17)$$

donde $N_{b,err}$ es la cantidad de bits erróneos y N_b es la longitud total del vector transmitido. A diferencia del SER, esta métrica evalúa el desempeño final a nivel de bits, por lo que también refleja el efecto del tipo de mapeo utilizado, ya sea Gray o binario.

4.6. Verificación de la implementación

Para verificar el funcionamiento del modulador se realizaron pruebas con M -PSK para $M = 2$, $M = 4$, $M = 8$ y $M = 16$. En cada caso se utilizó una misma entrada binaria controlada de longitud 24, armada de forma tal que aparezcan las distintas palabras de $k = \log_2(M)$ bits asociadas a cada orden de modulación. De esta manera se puede comprobar que la función `modularSimbolos` agrupa correctamente los bits, asigna cada palabra al símbolo correspondiente y genera la constelación esperada.

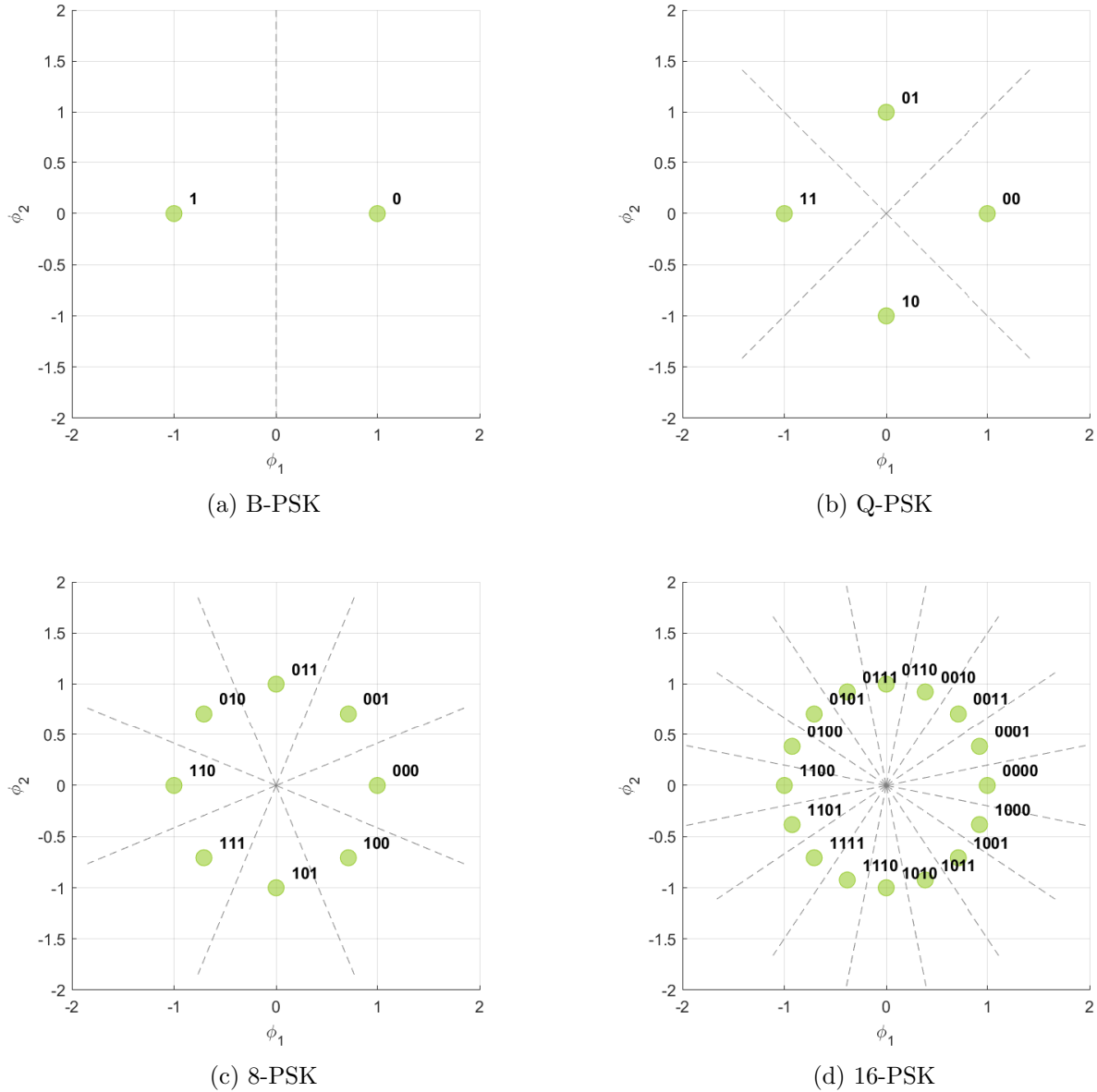


Figura 5: Constelaciones y fronteras de decisión generadas para las modulaciones M -PSK con $M = 2, 4, 8, 16$ con mapeo de Gray.

La figura 5 muestra las constelaciones obtenidas para los cuatro valores de M para el mapeo de Gray. En todos los casos los símbolos se ubican sobre una circunferencia, manteniendo amplitud constante y variando únicamente la fase de la portadora, tal como corresponde a una modulación PSK. Al aumentar M , la separación angular entre símbolos disminuye, por lo que la modulación

transporta más bits por símbolo pero se vuelve más sensible al ruido. Vemos como entre símbolos, se respeta el mapeo de Gray difiriendo un bit entre símbolos vecinos.

5. Codificación y decodificación de canal

Para la realización de la codificación y decodificación de canal se asignó la utilización de un Código Hamming extendido de parámetros $n = 15$ y $k = 11$ en función de la matriz generadora G de 11×15 presentada a continuación:

Matriz G asignada para realizar la codificación de Hamming Extendida

1	1	0	1	1	0	0	0	0	0	0	0	0	0	0
1	1	0	1	1	0	0	0	0	0	0	0	0	0	0
1	0	1	1	0	1	0	0	0	0	0	0	0	0	0
1	0	0	1	0	0	1	0	0	0	0	0	0	0	0
0	1	1	1	0	0	0	1	0	0	0	0	0	0	0
1	1	1	0	0	0	0	0	1	0	0	0	0	0	0
0	0	1	0	0	0	0	0	0	1	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1	0	0	0
1	1	0	0	0	0	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	0	0	0	0	0	1

5.1. El proceso de codificación de canal

Para poder realizar la codificación se fragmentará el mensaje obtenido de la codificación de fuente de a bloques de k bits. Para un mensaje saliente del bloque codificador de fuente, se irán tomando de a k bits, codificando esos agregando los $n - k$ bits de paridad por medio de la matriz generadora G , y se obtendrá una palabra final de n bits, con el mensaje original y los bits de paridad. Esto se puede visualizar claramente en la Figura 6, donde se identifican distintos bloques de tamaño k operándose con la matriz generadora y resultando en palabras de tamaño n .

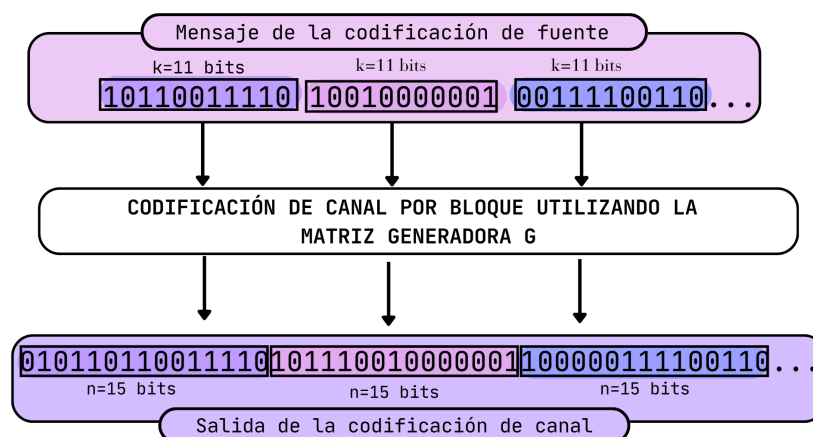


Figura 6: Diagrama explicativo de lo realizado en el módulo de codificación de canal.

El proceso realizado es el siguiente: se introduce a la codificación de canal por bloques un mensaje de $k = 11$ bits. Utilizando el ejemplo de la Figura 6, un mensaje que ingresa al codificador de canal, como podría ser la **10110011110**, se codificará por medio de la matriz generadora G , que tiene a su izquierda la matriz de paridad P (de 11×4) y a la derecha la matriz identidad I_{11} de 11×11 .

$$G = [P_{11 \times 4} | I_{11}] \quad (18)$$

Se generarán $n - k = 4$ bits de paridad utilizando G , que se añadirán al inicio de la palabra resultante, y serán seguidos por el mensaje original. En este caso, por ejemplo, por medio de la matriz generadora G se anexan los bits de paridad **0101** al mensaje original **10110011110**, obteniéndose a la salida una palabra de $n = 15$ bits como el siguiente: **010110110011110**.

5.2. Codificación de mensaje de k bits a una palabra de n bits a partir de la matriz generadora G asignada

Como ya se explicó previamente, por medio de la matriz generadora G se añadirán $n - k = 4$ bits de paridad al inicio del mensaje de $k = 11$ bits a codificar, lo que resultará en una palabra de $n = 15$ bits. Se transponen la matriz generadora G y el bloque de k bits a convertir y se realiza la multiplicación. Como al realizar esta operación se generan valores distintos de cero y uno (por la sumatoria del armado de paridades), luego se modularizan todos estos valores a ceros y unos (buscando el resto a 2 del valor obtenido) y se transpone el arreglo resultante para que sea de 1×15 . Esto puede apreciarse en la función presentada a continuación:

Codificación de mensaje de k bits (bloque) a palabra de n bits (bloque codificado)

```
function bloque_codificado = codificacion_Hamming_bloque(bloque, k, n, G)
    bloque_codificado = mod(G' * bloque', 2)';
end
```

5.3. División del arreglo completo en bloques de $k = 11$ bits y, por medio de la codificación de canal, obtención del arreglo de palabras de $n = 15$ bits codificadas

El primer gran inconveniente que se cuestionó al encarar este ítem fue qué se debía hacer en el caso de que los símbolos no fuesen múltiplos de k , de modo que se podría encontrar en el arreglo un último paquete de datos con menos de k bits. Para solucionar este inconveniente se realizó *zero-padding*, agregando ceros al final de los símbolos propios del mensaje para que la palabra a insertar a la codificación de canal fuese de k bits. De este modo, la cantidad final de ceros y unos a la salida del codificador de canal sería múltiplo de n bits, evitando problemas en el decodificador de canal y permitiendo realizar la codificación de canal apropiadamente por medio de la matriz generadora.

Se obtiene entonces el arreglo de ceros y unos saliente de la codificación de fuente por medio de la lectura del archivo de salida, realizada con la función `archivo_a_arreglo()`, que al recibir el nombre de un archivo de entrada genera un arreglo columna (es decir, con una sola fila) con ceros y unos leídos del archivo.

Luego, con la entrada a la codificación de canal ya en formato de arreglo, se llama a la función `codificador_canal()`; que recibe el arreglo, los valores de k y n , y la matriz generadora G . La función primero verifica si la cantidad de datos en la entrada es múltiplo de k y, si no es el caso, lo nota para dar espacio adicional en el arreglo de salida para los bits que quedan por fuera (con sus ceros agregados a posteriori). Se genera luego el arreglo completo (inicialmente todo en cero) para pedir la memoria que necesitará el arreglo completo una sola vez, en lugar de apendizar al mismo cada nueva palabra de n bits luego de codificar cada bloque de k bits, lo que requeriría pedir memoria

en cada iteración y resultaría costoso temporalmente para el programa. El tamaño total del arreglo será, dado que de cada k bits de entrada se tendrán n bits de salida; y considerando que se pueden añadir $extra = n$ bits en caso de que la división entre $size(arreglo_entrada)$ y k tenga resto no nulo; y $extra = 0$ de no ser el caso,

$$size(arreglo_salida) = extra + n * size(arreglo_entrada)/k \quad (19)$$

Luego, para cada bloque de k bits, se realiza lo siguiente: primero se separa el bloque de k bits a codificar utilizando la función `parsear_arreglo()`, que añade los ceros en el caso en el que el bloque enviado sea de tamaño menor a k , y devuelve siempre un arreglo de k bits. Éste luego se codifica con la función `codificacion_Hamming_bloque()` descrita anteriormente, añadiéndose al vector de salida de la codificación en la posición siguiente al bloque anterior.

5.4. Obtención de la matriz de verificación de paridad H a partir de los valores de k y n y de la matriz generadora G

Para conseguir la matriz de paridad, se puede expresar dicha matriz de la siguiente manera:

$$H = [I_4 | P^T] \quad (20)$$

Donde dicha matriz de paridad P es justamente la matriz mencionada y obtenida en la ecuación 18:

$$G = [P_{11 \times 4} | I_{11}] \quad (21)$$

Una vez definidas estas matrices y relaciones, a partir de la función `matrizParidad()` utilizando la matriz G y los valores n y k como parámetros, se obtiene la siguiente matriz de paridad H :

Matriz de Paridad H obtenida a partir de la matriz generadora G

```

1 0 0 0 1 1 1 0 1 0 0 1 1 0 0
0 1 0 0 1 0 0 1 1 0 1 0 1 1 0
0 0 1 0 0 1 0 1 1 1 0 0 0 1 0
0 0 0 1 1 1 1 1 0 0 0 0 0 0 1

```

5.5. Obtención de la tabla de síndromes S a partir de la matriz de chequeo de paridad H

Para obtener el síndrome de una palabra, se puede calcular de la siguiente manera:

$$s = rH^T \quad (22)$$

Dado que la matriz G es de tamaño $(15, 11)$ y $n - k = 4$, existen exactamente $2^4 = 16$ síndromes posibles. Siendo un síndrome donde no hay error "0000" y 15 síndromes para errores de un bit. Utilizando la función `tablaSindromes()`, ingresando la matriz de paridad H , resulta en la siguiente tabla:

Tabla de síndromes S obtenida a partir de la matriz de paridad H

0 0 0 0
1 0 0 0
0 1 0 0
0 0 1 0
0 0 0 1
1 1 0 1
1 0 1 1
1 0 0 1
0 1 1 1
1 1 1 0
0 0 1 0
0 1 0 0
1 0 0 0
1 1 0 0
0 1 1 0
0 0 0 1

5.6. Detección y corrección de errores ocurridos en una palabra de $n = 15$ bits por medio de la matriz de chequeo de paridad H y la tabla de síndromes S

Para observar la capacidad de detección y corrección de errores, se realizaron 3 ejemplos utilizando la función `corregir_palabra()`:

Vectores para el análisis de detección

1 0 0 1 0 1 1 0 1 1 0 0 0 1 1	Vector Original
1 0 1 1 0 1 1 0 1 1 0 0 0 1 1	Vector con 1 error
1 0 0 1 0 1 1 0 1 1 0 1 0 1 1	Vector con 1 error distinto
1 0 1 1 0 1 1 0 1 1 0 1 0 1 1	Vector con mas de 1 error

El primer vector es un mensaje codificado utilizando la matriz generadora de Hamming. Cuando ocurre un error en dicho vector, como se ve en la Tabla 5.6, utilizando la matriz H y la tabla S , se puede corregir ese error. Distinto es el caso cuando ocurre un error diferente, donde el vector, luego de utilizar el método anteriormente usado, permanece corrompido igualmente. Y por último, similar es el caso donde si se tiene más de 1 error, el vector permanece corrompido o, incluso, puede llegar a corromperse más dependiendo del error. Esto se debe a los parámetros de la matriz de Hamming que se encuentran en su sección dedicada. Para el análisis de decodificación de canal, se utiliza esta misma función dentro de una función denominada `decodificación_Hamming_bloque()`, donde se analiza parte de un mensaje más grande.

5.7. Decodificación de canal con detección y corrección de errores por medio de la matriz de paridad H y la tabla de síndromes S

Se obtiene el vector de ceros y unos saliente de la demodulación por medio de la lectura del archivo de entrada, realizada con la función `archivo_a_arreglo()`, que al recibir el nombre de un

archivo de entrada genera un arreglo columna (es decir, con una sola fila) de los ceros y unos leídos del archivo.

Luego, con la entrada a la codificación de canal ya en formato de arreglo, se llama a la función `decodificador_canal()`; que recibe el arreglo, los valores de k y n , y la matriz generadora G . La función no necesita verificar si la cantidad de datos en la entrada es múltiplo de n porque esto fue asegurado por la codificación de canal. Se genera un arreglo completo (inicialmente todo en cero) para pedir la memoria que necesitará el arreglo una sola vez, en lugar de apendizar al mismo cada nueva palabra de k bits luego de decodificar cada bloque de n bits. El tamaño total del arreglo será de:

$$size(arreglo_salida) = k * size(arreglo_entrada)/n \quad (23)$$

Luego, para cada bloque de n bits, se realiza lo siguiente: primero se separa el bloque de n bits a codificar utilizando la función `parsear_arreglo()`, que devuelve siempre un arreglo de n bits. Éste luego se decodifica con la función `decodificacion_Hamming_bloque()` descrita anteriormente, añadiéndose al vector de salida de la decodificación en la posición siguiente al bloque anterior.

5.8. Cálculo de distancia mínima d_{min} , cantidad máxima de errores a detectar e y cantidad máxima de errores a corregir t

Para el cálculo de la distancia mínima, se generan todos los mensajes posibles, siendo estos una cantidad total de 2^k siendo $k = 11$, y se codifican por medio de la matriz generadora, obteniéndose 2^k palabras de $n = 15$ bits. Se evalúa el peso de cada palabra codificada, sumando la cantidad de unos de la misma como se ve en la ecuación 24, siendo x_j el bit en la posición j de la palabra siendo analizada.

$$Peso_i = \sum_{j=1}^n x_j; i \in [1, 2^k] \quad (24)$$

Se busca el código de menor peso; es decir, $\min_{Peso_i}$; siendo este un $d_{min} = 2$.

Para el cálculo de la cantidad de errores detectables e , resulta importante notar que “un bloque de código con distancia mínima d_{min} garantiza que todos los patrones de error con $d_{min} - 1$ o menos errores podrán ser detectados”;¹ por ende:

$$e = d_{min} - 1 = 1 \quad (25)$$

Para el cálculo de la cantidad de errores corregibles t , que corresponde a “la cantidad máxima de errores por palabra codificada para los cuales su corrección es garantizable”², se realiza el siguiente cálculo, donde $\lfloor x \rfloor$ representa el mayor entero que no exceda x ; es decir, un redondeo al piso en lugar de al techo del número flotante:

$$t = \lfloor \frac{d_{min} - 1}{2} \rfloor = 0 \quad (26)$$

Para un código de parámetro t por lo general se podrán corregir hasta 2^{n-k} patrones de error; en este caso, $2^4 = 16$ patrones de error.

Estos parámetros son vitales para identificar que la matriz generadora G de Hamming, a partir de su matriz de paridad y tabla de síndromes, genera códigos para los cuales se puede garantizar la detección de al menos un error, y no se puede asegurar la corrección de ningún potencial error.

¹“A block code with minimum distance d_{min} guarantees that all error patterns of $d_{min}-1$ or fewer errors can be detected”(EL LIBRO, p.345)

²“the error-correcting capability t of a code is defined as the maximum number of guaranteed correctable errors per codeword”(EL LIBRO, p.334)

6. Efectos del canal

Es de suma importancia modelar los efectos del canal en nuestro sistema, para entender como afectan situaciones como el ruido térmico o la atenuación de la señal a la probabilidad de error del sistema. Ac continuación se modelará el ruido térmico y la atenuación del canal con 2 funciones de matlab, para luego graficar y comparar los resultados.

6.1. Generación de muestra aleatoria de ruido térmico

Para modelar el ruido térmico se utilizó una muestra aleatoria gaussiana de media nula. La función implementada fue `ruido=generacion_ruido_termico(v,N)`, donde `v` es la varianza del ruido $N_0/2$ y `N` es la cantidad de muestras a generar. En MATLAB se implementó como

$$\text{ruido} = \text{sqrt}(v) * \text{randn}(1, N). \quad (27)$$

Como `randn(1,N)` genera una secuencia de variables aleatorias con distribución normal estándar, $Z \sim \mathcal{N}(0, 1)$, al multiplicarla por $\sqrt{v}$ se obtiene una muestra con varianza v :

$$n = \sqrt{v} Z, \quad n \sim \mathcal{N}(0, v) \quad (28)$$

Para definir la varianza a partir de una relación señal a ruido deseada, se fijó $SNR = 6$ dB y llevó a escala lineal. Luego, usando la energía media por símbolo estimada previamente, se calculó

$$N_0 = \frac{\bar{E}_s}{SNR} \quad (29)$$

De esta manera, el ruido agregado queda asociado directamente con la energía de los símbolos modulados y con la relación señal a ruido especificada para la simulación.

6.2. Generación de atenuación aleatoria del canal

Además del ruido térmico, se modeló una atenuación aleatoria del canal mediante la función `atenuacion=generacion_atenuacion(min,max)`. Esta función genera un único valor aleatorio con distribución uniforme en el intervalo definido por `min` y `max`. En MATLAB se implementó como

$$\text{atenuacion} = \text{min} + (\text{max}-\text{min}) * \text{rand}() \quad (30)$$

Como `rand()` genera una variable aleatoria uniforme entre 0 y 1, la expresión anterior desplaza y escala ese valor para obtener

$$a \sim \mathcal{U}(\text{min}, \text{max}) \quad (31)$$

En las simulaciones se utiliza el mismo valor de atenuación para toda la secuencia transmitida. Es decir, no se genera una atenuación distinta para cada símbolo, sino que se modela un canal con ganancia constante durante todo el bloque de datos. Si s_i representa el símbolo transmitido, el efecto de la atenuación se expresa como

$$s_{i,at} = a s_i, \quad (32)$$

donde a es el factor de atenuación generado. De esta forma, todos los puntos de la constelación se acercan al origen en la misma proporción, manteniendo su forma relativa pero reduciendo la energía recibida.

6.3. Verificación de la implementación

Para verificar el funcionamiento de las funciones de canal, se utilizó una entrada binaria controlada de 192 bits. Esta longitud es múltiplo de $k = \log_2(M)$ para BPSK, QPSK, 8-PSK y 16-PSK, por lo que permite modular la misma secuencia en los cuatro órdenes considerados. En particular, se obtienen 192 símbolos para BPSK, 96 para QPSK, 64 para 8-PSK y 48 para 16-PSK.

Luego de modular la secuencia con mapeo Gray, se aplicaron los efectos del canal: ruido térmico AWGN con $SNR = 6$ dB y una atenuación constante para todo el bloque transmitido. Finalmente, los símbolos afectados por el canal se ingresaron al receptor para observar cómo se desplazan y dispersan respecto de las constelaciones ideales. Las constelaciones recibidas se muestran en la figura 7.

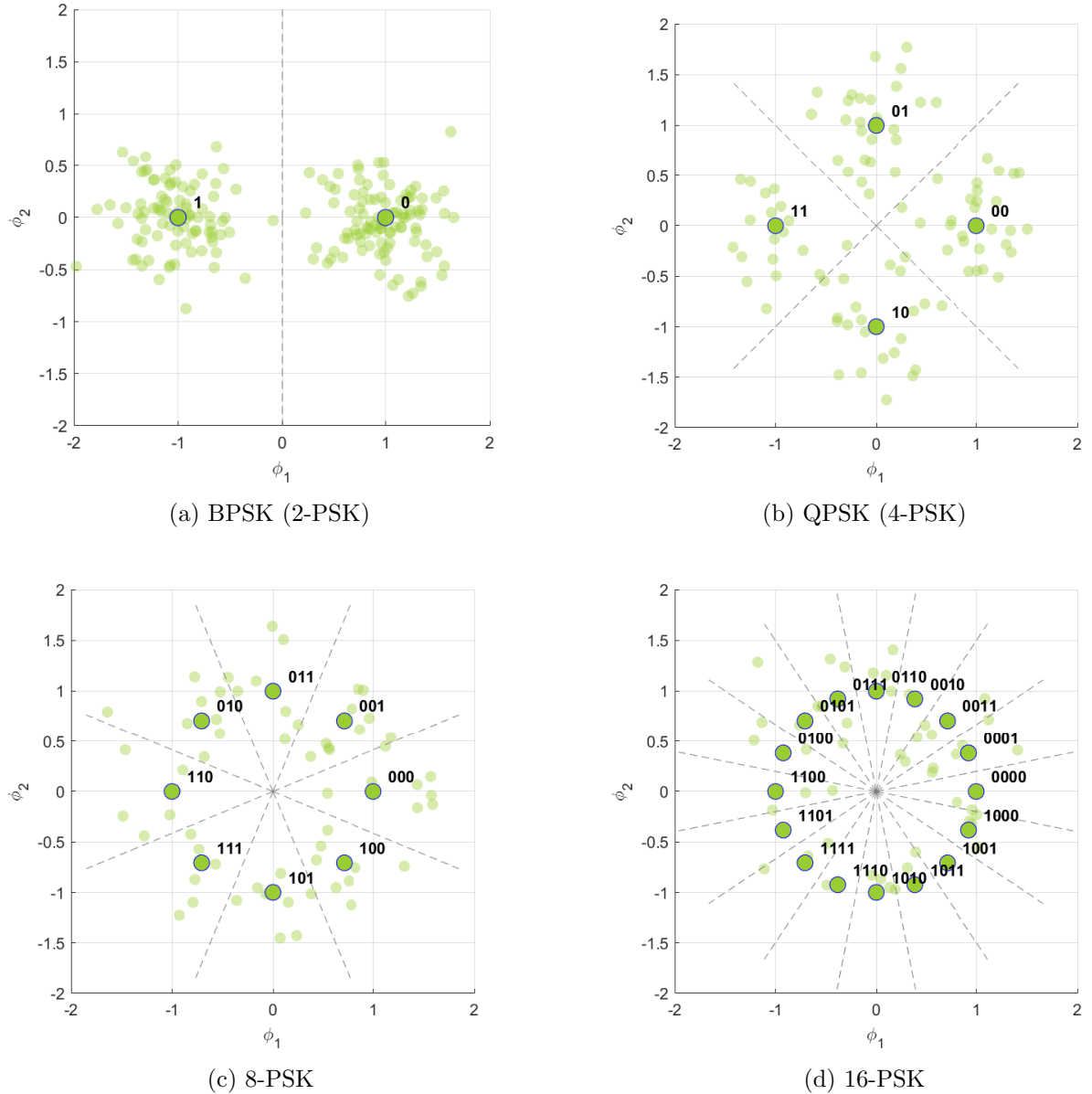


Figura 7: Constelaciones y fronteras de decisión generadas para BPSK, QPSK, 8-PSK y 16-PSK con mapeo Gray y $SNR = 6$ dB.

7. Verificación del sistema

Para verificar el funcionamiento correcto del sistema se realizaron las siguientes pruebas:

7.1. Verificación de la etapa de codificación de fuente

Para comprobar que la codificación de fuente se estaba realizando correctamente, se realizó una función de prueba por la cual se ingresara un archivo de texto, se codificara por medio de la codificación de fuente y luego se decodificara por medio del decodificador de fuente. Para esta función, llamada `prueba_cod_fuente_directo_decod_fuente()`, recibe el nombre del archivo de entrada y del de salida. Por medio de la función `comparar_entrada_y_salida()` se comparan luego estos dos archivos para verificar si la entrada y la salida diferían.

7.2. Verificación de la etapa de codificación de canal

Para verificar el correcto funcionamiento de la codificación de canal, se realizó una serie de pruebas denominadas `prueba_decod_cod_canal` donde se le envía un arreglo de entrada de tamaño mayor a 11 para utilizar las funciones del codificador en su totalidad. Una vez realizada dicha codificación, se lo envía directamente al decodificador para obtener el arreglo original enviado. Una vez obtenida la salida del decodificador, se compara el vector original con el vector decodificado, elemento por elemento, para corroborar el correcto funcionamiento.

7.3. Verificación de la etapa de modulación y demodulación con efectos del canal

Para corroborar la correcta implementación de la modulación, la demodulación y los efectos del canal se creó la prueba `prueba_modulador_directo_demodulador()` que recibe los parámetros de la modulación y demodulación como el tipo de mapeo (Gray o binario) y de modulación (M-PSK / M-FSK en nuestro caso), el orden M de la modulación, una entrada en formato de arreglo (que hipotéticamente sería la salida de la etapa de codificación de canal), los valores máximos y mínimos para la atenuación, el ratio de ruido a señal (*signal to noise ratio*) y booleanos para indicar si se debe aplicar la atenuación y/o el ruido térmico en la prueba a realizar.

Por medio de la función de prueba se verifica el error de bit por símbolo y se mapean las constelaciones de modulación y demodulación.

7.4. Verificación conjunta de la etapa de codificación de fuente y codificación de canal

Luego de haber comprobado el correcto funcionamiento entre módulos de la misma etapa entre transmisor y receptor, se procedió a juntar los distintos módulos, iniciando por la codificación de fuente y la codificación de canal. Para esto se creó la prueba `prueba_cod_decod_fuente_canal()`, mediante la cual se verificó que la entrada al codificador de fuente y la salida del decodificador de fuente, pasando por la etapa de codificación y decodificación de canal, no tuviesen diferencias.

Primero se utilizó el archivo `'entrada.txt'`, que contiene el texto de bienvenida del Campus Virtual de la materia utilizado en la prueba de codificación y decodificación de canal; y luego se

7.5. Verificación conjunta de todo el sistema

Una vez realizadas las verificaciones modulares por etapas, se implementó una última prueba global que verifica el correcto funcionamiento del sistema en su totalidad. Para esto se juntaron todas las etapas del sistema en funciones separadas y se armó una función de prueba denominada `prueba_programa_completo()`. Se le envió un archivo de entrada del tipo `.txt` para que, una vez

finalizado el recorrido completo del sistema, se verificara que la salida, nuevamente del tipo `.txt`, coincidiera con la entrada.

8. Conclusión